Zurich University of Applied Sciences

School of Management and Law

Prototyping – Vorlesung 6
SvelteKit Teil 4: Detailseiten, load-Funktion, Form Actions, DB-Anbindung



Building Competence. Crossing Borders.

Max Meisterhans
max.meisterhans@zhaw.ch

Lernziele

- Ihr könnt mit SvelteKit Detailseiten erstellen.
- Ihr könnt mit SvelteKit Daten verwalten (mit einer MongoDB).

Load-Funktion und Pfadparameter

Load-Funktion

Load-Funktionen können verwendet werden, um einer Svelte-Komponente Daten zur Verfügung zu stellen.

*artists/**+page.server.js***

```
export async function load() {
    return {
        artists: [
            {
                _id: 'One',
                name: 'Daft Punk',
                origin: 'France'
            },
            {
                _id: 'Two',
                name: 'Chilly Gonzalez',
                origin: 'Canada'
            }
        ]
    }
}
```

*artists/**+page.svelte***

```
<script>
  let { data } = $props();
</script>

<ul>
  {#each data.artists as artist}
    <li>{artist.name}</li>
  {/each}
</ul>
```

- Daft Punk
- Chilly Gonzalez

https://svelte.dev/docs/kit/load

Pfadparameter

Wie werden *dynamische Routen* verwendet?

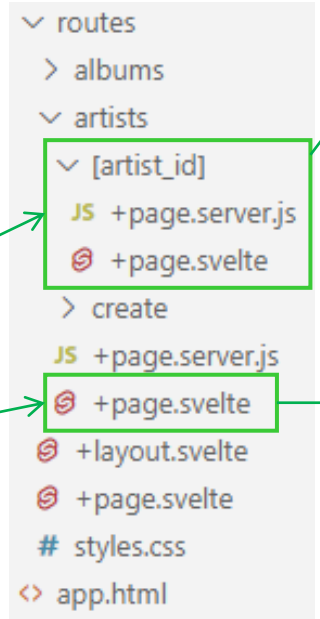
- `/artists/one` → Detailseite für Artist one
- `/artists/two` → Detailseite für Artist two
- …

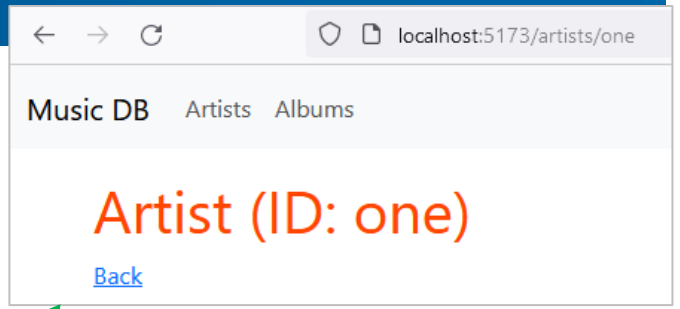
Pfadparameter werden in SvelteKit über die Ordnerstruktur festgelegt.

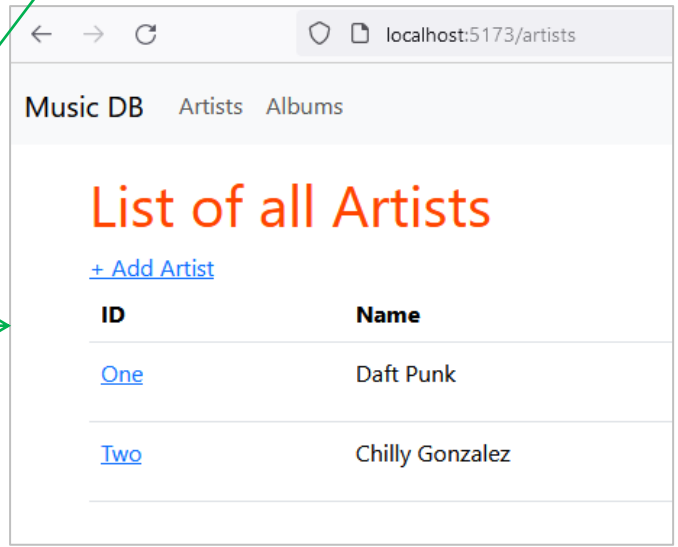
```
<a href="/artists/one">One</a>
```

```
<a href="/artists">Artists</a>
```

https://svelte.dev/docs/kit/routing







Pfadparameter

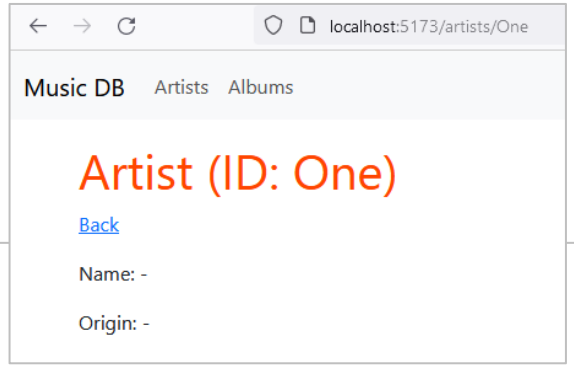
Auf den Pfadparameter kann in der Load-Funktion wie folgt zugegriffen werden:

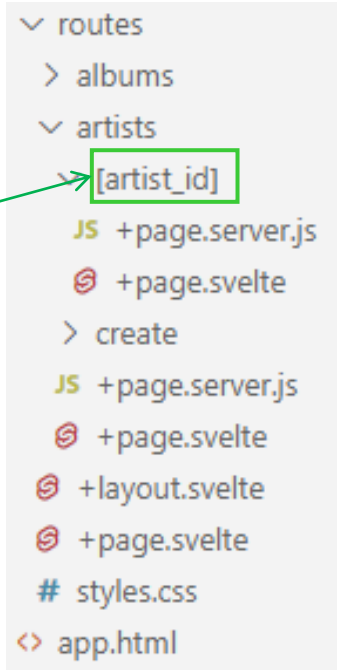
```
http://localhost:5173/artists/one
```

```
export async function load({ params }) {

    return {
        artist: {
            _id: params.artist_id,
            name: "-",
            origin: "-"
        }
    };
}
```

*/artists/[artist_id]/**+page.server.js***





https://svelte.dev/docs/kit/load#Page-data

Daten aus einer MongoDB laden

Node.js und NPM

Node.js

- Laufzeitumgebung, um JavaScript ohne Browser auszuführen.
- Gut geeignet für Serveranwendungen (z.B. SvelteKit Server).
- www.nodejs.org



NPM

- JavaScript Package Manager
- Wird verwendet, um Abhängigkeiten (Libraries) in Node-Anwendungen zu installieren und zu verwalten.
- Wird als Teil von Node.js installiert.
- www.npmjs.com



MongoDB Driver für Node.js / SvelteKit

Package «mongodb»

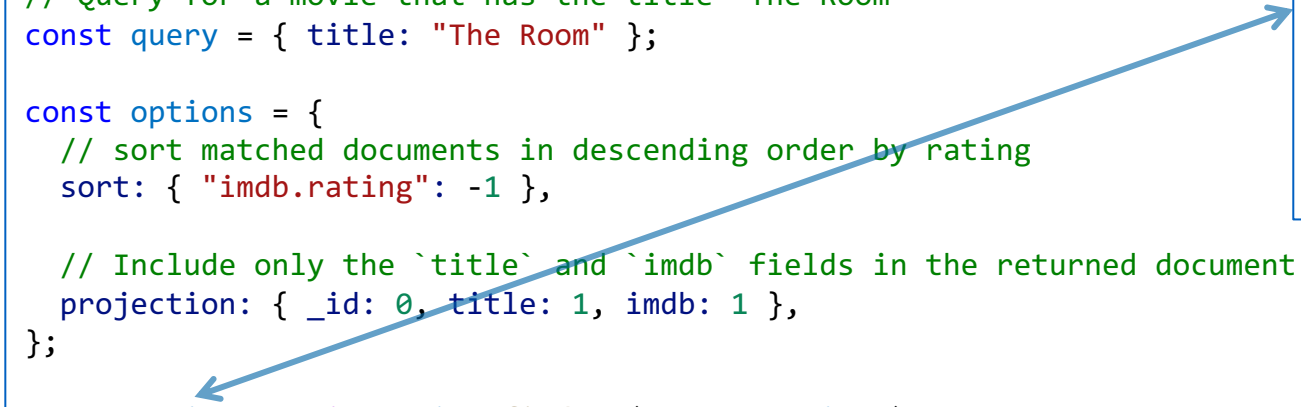
- Ein Node.js-Driver für MongoDB. Dokumentation:
  - https://www.npmjs.com/package/mongodb
  - https://www.mongodb.com/docs/drivers/node/current/

```
// Query for a movie that has the title 'The Room'
const query = { title: "The Room" };

const options = {
  // sort matched documents in descending order by rating
  sort: { "imdb.rating": -1 },

  // Include only the `title` and `imdb` fields in the returned document
  projection: { _id: 0, title: 1, imdb: 1 },
};

const movie = await movies.findOne(query, options);
```



```
{
  title: 'The Room',
  imdb: {
    rating: 3.5,
    votes: 25673,
    id: 368226
  }
}
```

MongoDB in SvelteKit

*lib/server/**db.js***

```
import { MongoClient, ObjectId } from 'mongodb';
import { DB_URI } from '$app/env/private';

const client = new MongoClient(DB_URI);
await client.connect();
const db = client.db('MusicDB') // select database

async function getArtists() {
    let artists = [];
    try {
        const collection = db.collection('artists');
        const query = {};
        artists = await collection.find(query).toArray();
        artists.forEach(artist => {
            // convert ObjectId to String
            artist._id = artist._id.toString();
        });
    } catch (error) {
        // TODO: errorhandling
    }
    return artists;
}

// export functions so that they can be used in other files
export default { getArtists }
```

In der Datei *.env* können Umgebungsvariablen (z.B. Datenbank-Credentials) gespeichert werden.

.env

```
DB_URI=mongodb+srv://...
```

src/env.js

```
import { defineEnvVars } from '@sveltejs/kit/env';

export const variables = defineEnvVars({
    DB_URI: {}
});
```

*routes/artists/**+page.server.js***

```
import db from "$lib/db.js"

export async function load() {

    return {
        artists: await db.getArtists()
    }
}
```

Asynchrone Funktionen, async/await

JavaScript-Funktionen können *asynchron* sein. Typisches Beispiel: Abrufen von Daten via Netzwerkverbindung, z.B. HTTP-Request. Asynchrone Funktionsaufrufe blockieren die App nicht.
Beispiel: Bei einem HTTP-Request oder einer Datenbankabfrage können trotzdem andere Funktionen aufgerufen werden, während auf die Antwort des Servers gewartet wird.

Wenn *await* in einer Funktion verwendet wird, muss die Funktion als *async* deklariert werden

Asynchroner Funktionsaufruf

*lib/server/**db.js***

```
async function getArtists() {
    // ...
    artists = await collection.find(query).toArray();
    // ...
    return artists;
}
```

*routes/artists/**+page.server.js***

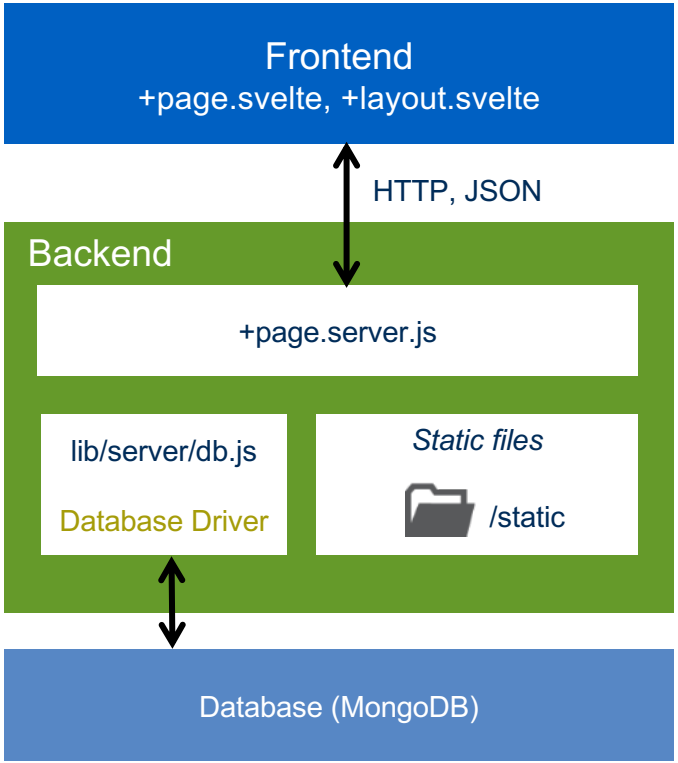
```
import db from "#lib/server/db.js"

export async function load() {

    return {
        artists: await db.getArtists()
    }
}
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

Zusammenfassung: Architektur einer SvelteKit-App

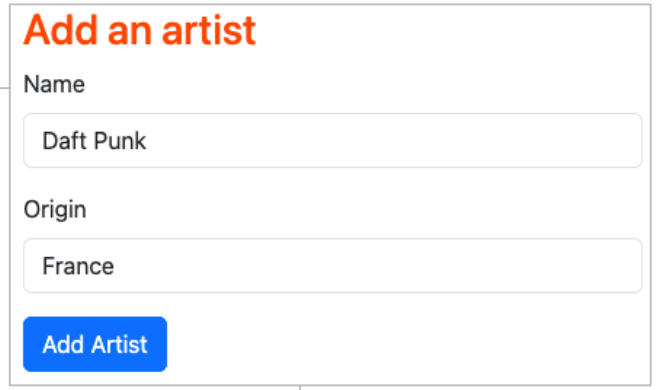


Daten via *Form actions* in einer MongoDB verwalten

Formulare

Formulare sind der «klassische» Weg, um mit einer Webanwendung Daten zu verwalten:

```
<form method="POST" action="?/create">
  <div class="mb-3">
    <label for="" class="form-label">Name</label>
    <input name="name" class="form-control" type="text" />
  </div>
  <div class="mb-3">
    <label for="" class="form-label">Origin</label>
    <input name="origin" class="form-control" type="text" />
  </div>
  <button type="submit" class="btn btn-primary"> Add Artist </button>
</form>
```



Formulare werden «submitted», meist via Button → eingegebene Daten werden verarbeitet.

Form Actions

Neben der *load*-Funktion, die verwendet wird um Daten zu laden, können zusätzlich *actions* definiert werden, um beispielsweise Daten zu erstellen, zu aktualisieren oder zu löschen.

Name der Action, die bei einem Submit ausgeführt wird.

*artists/create/**+page.svelte***

```
<form method="POST" action="?/create">
  <div class="mb-3">
    <label for="" class="form-label">Name</label>
    <input name="name" class="for
  </div>
  <div class="mb-3">
    <label for="" class="form-label">Origin</label>
    <input name="origin" class="form-control" type="text" />
  </div>
  <button type="submit" class="btn btn-primary">Add Artist</button>
</form>
```

Das Attribut «name» bestimmt, wie in page.server.js auf die Inputdaten zugegriffen werden kann.

*artists/create/**+page.server.js***

```
export const actions = {
  create: async ({request}) => {
    const data = await request.formData();
    let artist = {
      name: data.get("name"),
      origin: data.get("origin"),
    }
    await db.createArtist(artist);
  }
}
```

```
{
  name: "Daft Punk",
  origin: "France"
}
```

https://svelte.dev/docs/kit/form-actions
https://svelte.dev/docs/kit/form-actions#Named-actions

Feedback anzeigen

Wenn nach Ausführung einer Action ein Feedback angezeigt werden soll (z.B. «Artist created»), kann diese ein Objekt mit beliebigen Daten zurückgeben, das dann auf der Page verarbeitet werden kann.

`form` als Prop deklarieren, damit auf den Rückgabewert der Action zugegriffen werden kann.

*artists/create/**+page.svelte***

```
<script>
  let { form } = $props();
</script>

<form method="POST" action="?/create">
<!-- ... -->
</form>

{#if form?.success}
  <p>Artist created</p>
{/if}
```

*artists/create/**+page.server.js***

```
import db from '#lib/server/db.js';

export const actions = {
  create: async ({request}) => {
    // ...
    return { success: true }
  }
}
```

Das Fragezeichen ist nötig, damit es keinen Fehler gibt, wenn die Seite neu geladen wird. Zu diesem Zeitpunkt existiert das Objekt `form` noch nicht.

https://svelte.dev/docs/kit/form-actions#Anatomy-of-an-action

Form Actions mit versteckten Argumenten (z.B. für ID)

Jeder Delete-Button ist eigentlich ein Formular mit einem versteckten Input-Element, in dem die ID des Artists steht.

Manchmal ist es nötig, einer Form Action Argumente mitzugeben, die für den User nicht sichtbar sind. Eine einfache Lösung ist die Verwendung versteckter Elemente.

Name	Origin	
Rihanna	United States	x
Boys Noize	Germany	x
Daft Punk	France	x
Chilly Gonzalez	Canada	x
Justin Bieber	Canada	x

*artists/**+page.svelte***

```
<tbody>
  {#each data.artists as artist}
    <tr>
      <!-- ... -->
      <td>
        <form method="POST" action="?/delete">
          <input type="hidden" name="id" value={artist._id} />
          <button class="btn btn-danger">X</button>
        </form>
      </td>
    </tr>
  {/each}
</tbody>
```

«Trick»: Verstecktes Input-Element (type="hidden") das als Wert die ID enthält.

*artists/**+page.server.js***

```
//...
export const actions = {
  delete: async ({request}) => {
    const data = await request.formData();
    await db.deleteArtist(data.get("id"));
  }
}
```

```
{
  id: 1234,
  name: "Daft Punk",
  origin: "France"
}
```

https://svelte.dev/tutorial/kit/named-form-actions